



**Department of Applied Computing and Artificial Intelligence
Faculty of Computing
UNIVERSITI TEKNOLOGI MALAYSIA**

**PROGRAMMING TECHNIQUE II
(SECJ1023)**

SEMESTER II 2023/2024

**Project and Assignment Software Documentation
RESTAURANT ORDERING//PAYMENT System**

**By
ELIJAH SHE YU SHENG (A23CS0073)
TAN ZHI MING (A23CS0189)
NG YU HIN (A23CS0148)
SECTION 02**

**Lecturer:
Ts. Dr. Johanna binti Ahmad
Date
28 June 2024**

TABLE OF CONTENT

PART 1: INTRODUCTION	
1.1 SYNOPSIS PROJECT	3
1.2 OBJECTIVE OF THE PROJECT	3
PART 2: SYSTEM ANALYSIS AND DESIGN (USE CASE, FLOWCHART,AND CLASS DIAGRAM)	5
2.1 SYSTEM REQUIREMENTS DETAIL DESCRIPTION FOR EACH USE CASES	6
2.2 SYSTEM DESIGN	
PART 3 SYSTEM PROTOTYPE	7-16
PART 4: APPENDIX	17-44

PART 1: INTRODUCTION

1.1 SYNOPSIS PROJECT

The Restaurant Transaction System is a software solution designed to automate the ordering and payment processes for customers in restaurants. Developed using Object-Oriented Programming (OOP) principles like inheritance, encapsulation, polymorphism, and data hiding, this system aims to address the challenges faced by both customers and merchants in managing order records and menu details effectively. Users can log in as either a customer or a merchant. The merchant menu allows for modifying dish details, editing payment details and setting up menu through various functions. On the customer side, the menu enables viewing meal details, placing orders, making payments and registering to become a member seamlessly. The system's user interface will be intuitive and user-friendly, facilitating quick and easy ordering and payments for customers. The Restaurant Ordering System aims to provide a comprehensive solution for efficient ordering and payment processes, thereby saving time and allowing customers to focus more on exploring meal details.

1.2 OBJECTIVE OF THE PROJECT

The primary objective of the Restaurant Ordering System is to enhance the efficiency of order management and payment processes within a restaurant environment through the use of Object-Oriented Programming (OOP) principles. By implementing features such as inheritance, encapsulation, polymorphism, and data hiding, the system will offer a robust solution for both customers and merchants. The system aims to provide a seamless experience for customers to place orders and make payments, while enabling merchants to manage dish details, setting up menus, and editing payment details efficiently. The ultimate goal is to deliver a comprehensive and user-friendly platform that improves operational efficiency and enhances the dining experience for customers.

PART 2: SYSTEM ANALYSIS AND DESIGN (USE CASE, FLOWCHART, AND CLASS DIAGRAM)

2.1 SYSTEM REQUIREMENTS

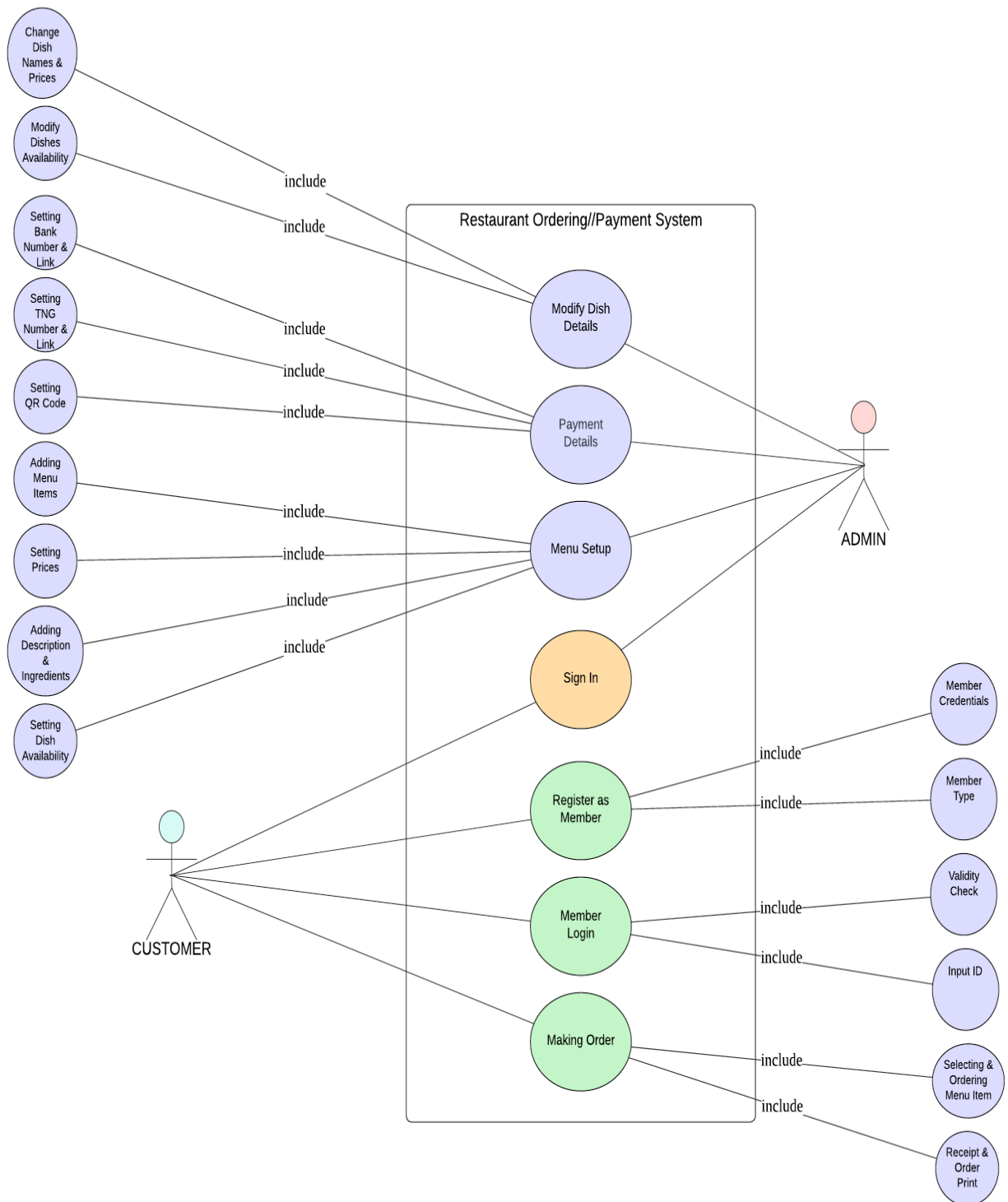


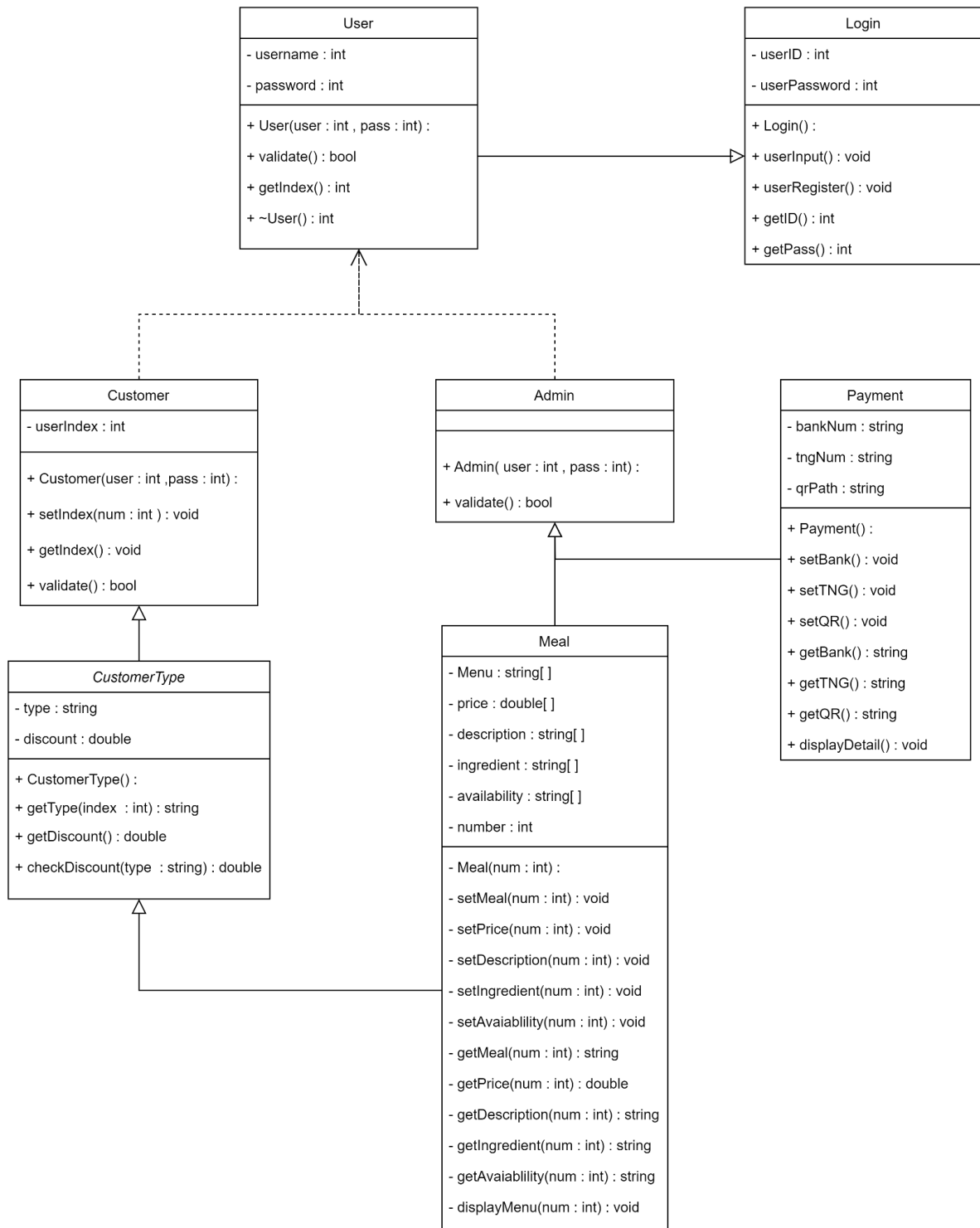
Figure 1 : Use Case Diagram for Restaurant Ordering/Payment System

DETAIL DESCRIPTION FOR EACH USE CASES

This system consists of 7 main use cases

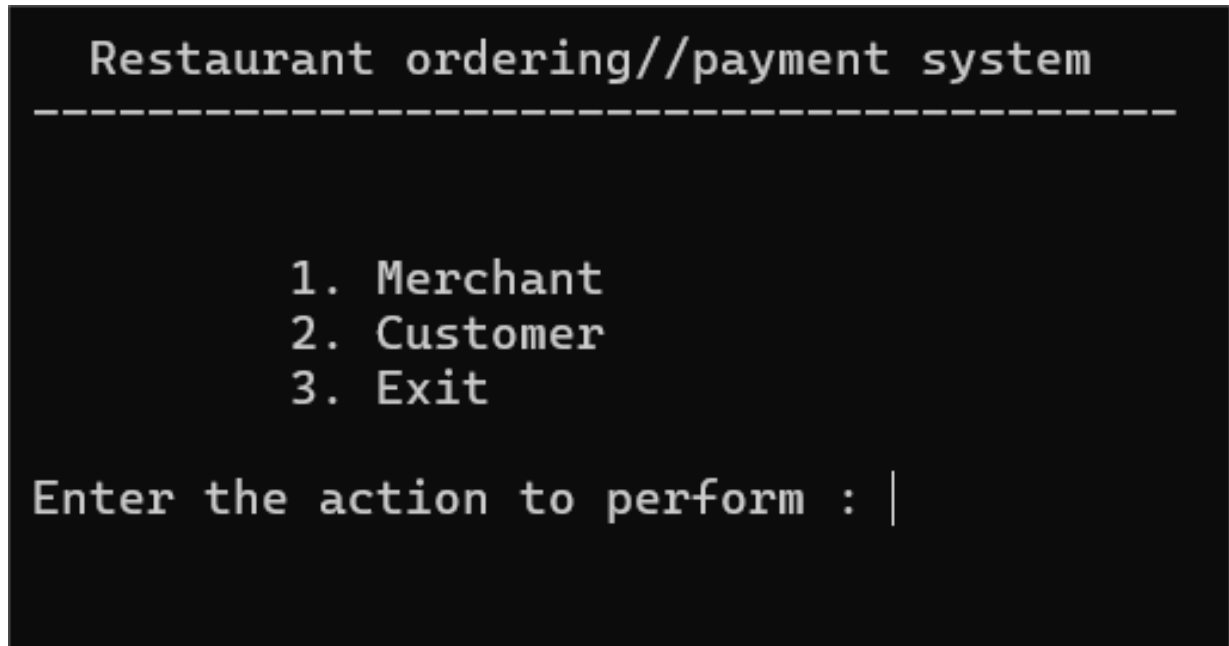
Use Case	Purpose
Modify Dish Details	To modify and update the menu items of each dish about each of its own name,price,and availability.
Payment Details	To set up the payment methods for convenience of the customers through bank,scanning QR code or TNG(TouchNGo)
Menu Setup	To add menu items to the daily menu,while setting each of its own price,description,ingredients used and availability which will be displayed in the menu for the customers to order.
Sign in	To ease the use of admin or customer when using the system, which can be instantly updated by the admin and displayed in the customer's menu
Register as Member	To sign up as a member, which can lead to customers having benefits such as discounts when ordering their favourite menu items with terms and conditions applied.
Member Login	To determine and verify whether or not the customer is a member to enjoy the benefits of the restaurant system.
Making Order	To place orders when using the system and receipt of the order will be printed and displayed to help the customer check whether their order was successful and correct as well as avoiding disorder.

2.2 SYSTEM DESIGN



PART 3 SYSTEM PROTOTYPE

1.LOGIN INTERFACE

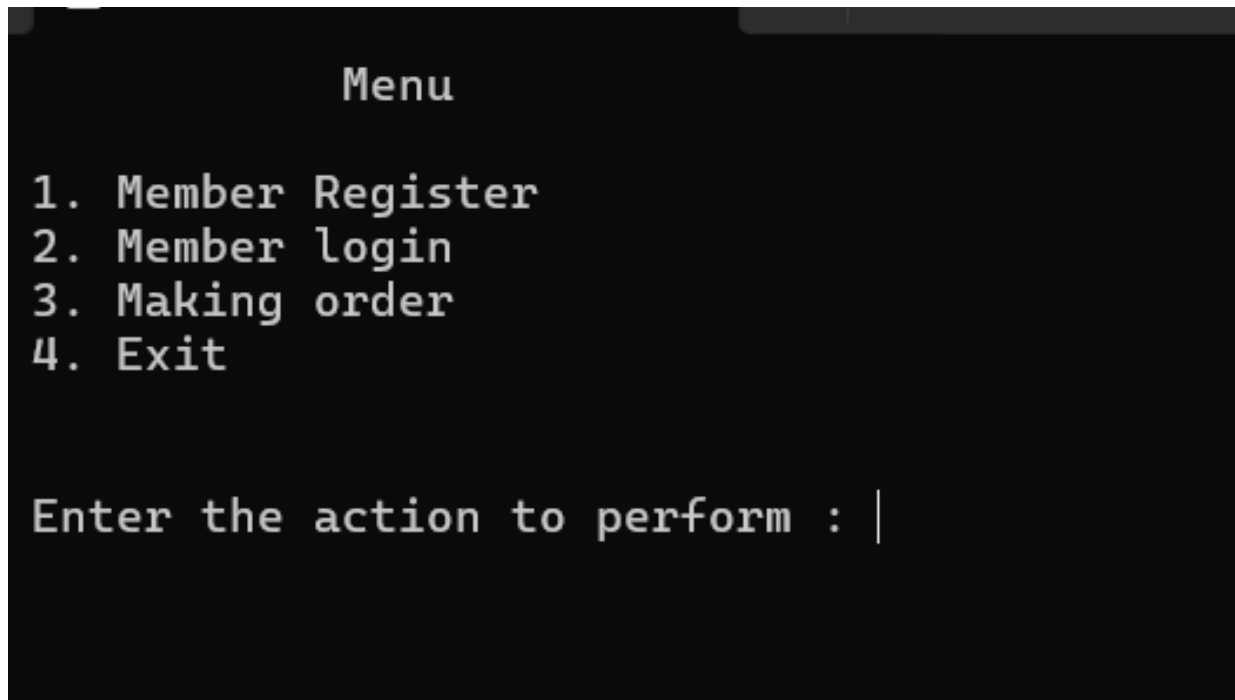


Screen 1: Login Interface

Screen 1: The admin must select 1 for Admin Login, while the customer must select 2 for customer Menu or 3 to Exit. Input must be an integer from 1-3; invalid entries will prompt an error and the screen reappear.

CUSTOMER INTERFACE

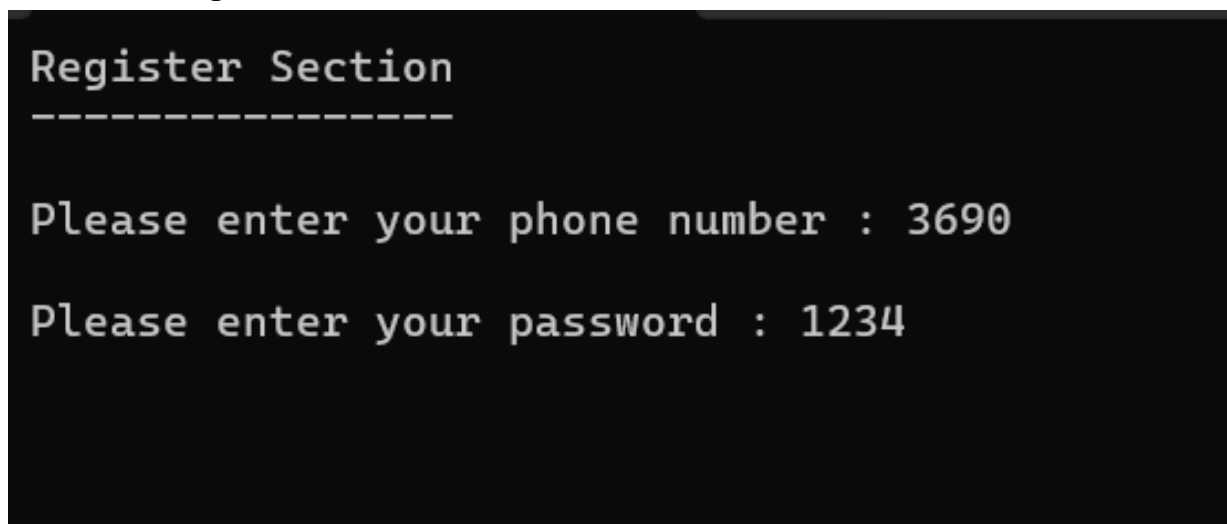
1.Customer Menu



Screen 2: Main Menu for Customer Interface

Screen 2: The customer is presented with 4 choices to choose which are member registration, member login, making an order as a guest or exit the menu.

2.Customer Registration



Screen 3: Customer Registration

Screen 3: During this interface, the customer must enter their phone number as customer id. Then, enter the password. The customer id and password can be referred to txt file name customerList.txt and customerPass.txt .

3.Customer Login

```
-----  
Member LOG IN  
-----  
  
Please enter your ID : 3690  
  
Please enter your password : 1234
```

Screen 4: Customer Login

Screen 4: The customer must login the customer id same as the registration one. Then, the password key must also be the same. If wrong, the id and password are required to enter again until the id and password is matched.

4.Customer Making Order

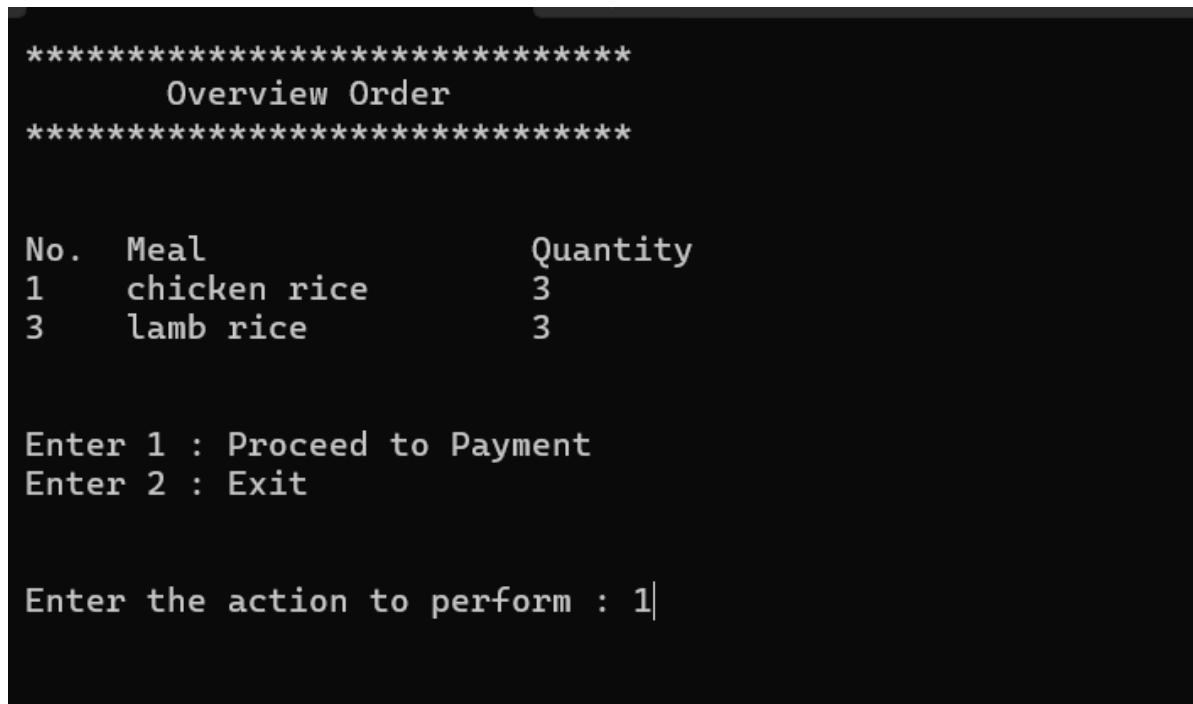
```
**Updated Menu**  
No.    Meal name      Price($)  Description      Ingredients      Availability  
1      chicken rice   7         not spicy       rice and chicken breast  Yes  
2      pork rice      8         not spicy       rice and pork pork      No  
3      lamb rice       9         not spicy       lamb and rice           Yes  
  
Enter the meal you want to add : 1  
How many do you want to order chicken rice : 3  
Do you want to add on something (Y)/(N) : Y  
  
Enter the meal you want to add : 2  
This meal is currently not available.  
  
Enter the meal you want to add : 3  
How many do you want to order lamb rice : 3  
Do you want to add on something (Y)/(N) : N|
```

Screen 5: Making the order

During this interface, the updated menu will be shown before the customer makes their own meal. The meal number must be entered to select their favourite food. If entered other than number, the invalid output will be shown and customers are required to enter the number again until the input is matched with the system. If the customer still wants to order another

type of food , the customer must enter “Y” to continue their order. If not, they must enter “N” to change to the payment interface. If a customer enters the unavailable meal, the “The meal is currently not available” message will come out.

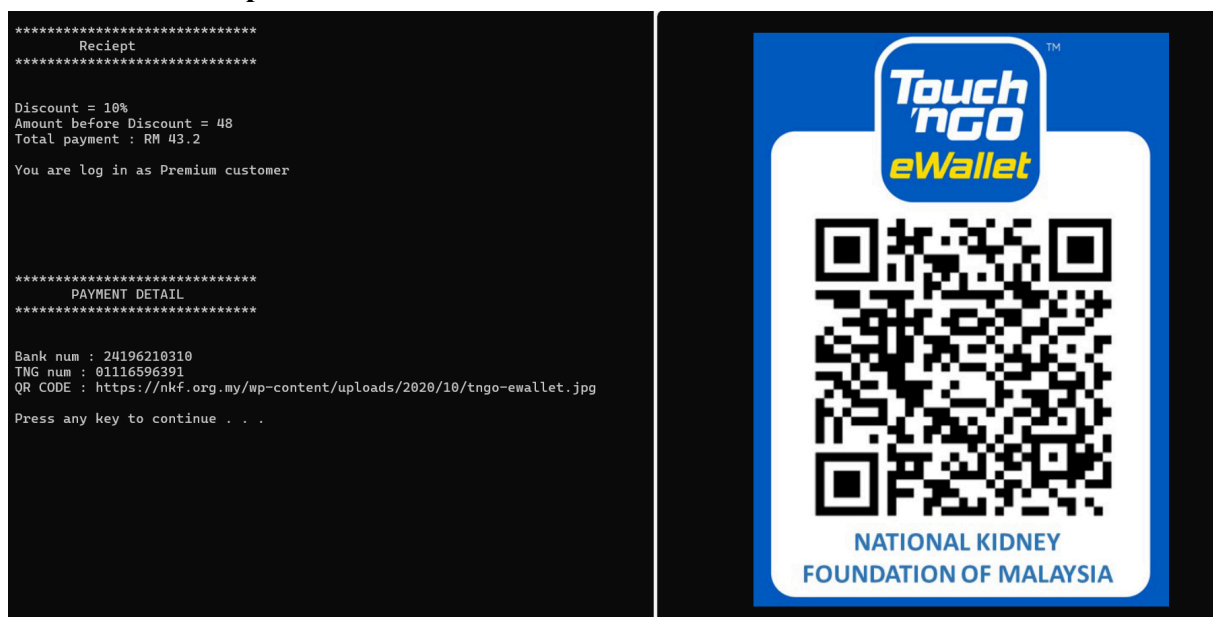
5.Customer Overviewing Order



Screen 6: Overview order

Screen 6: Before proceeding with their payment, the overview of their order will be shown to confirm meals selected are correct and quantities are also correct. The customer must enter 1 to continue payment or enter 2 to exit the payment interface. Other than 1 and 2, other input will be considered as wrong input until the correct input is matched with the system requirement.

6.Customer Receipt

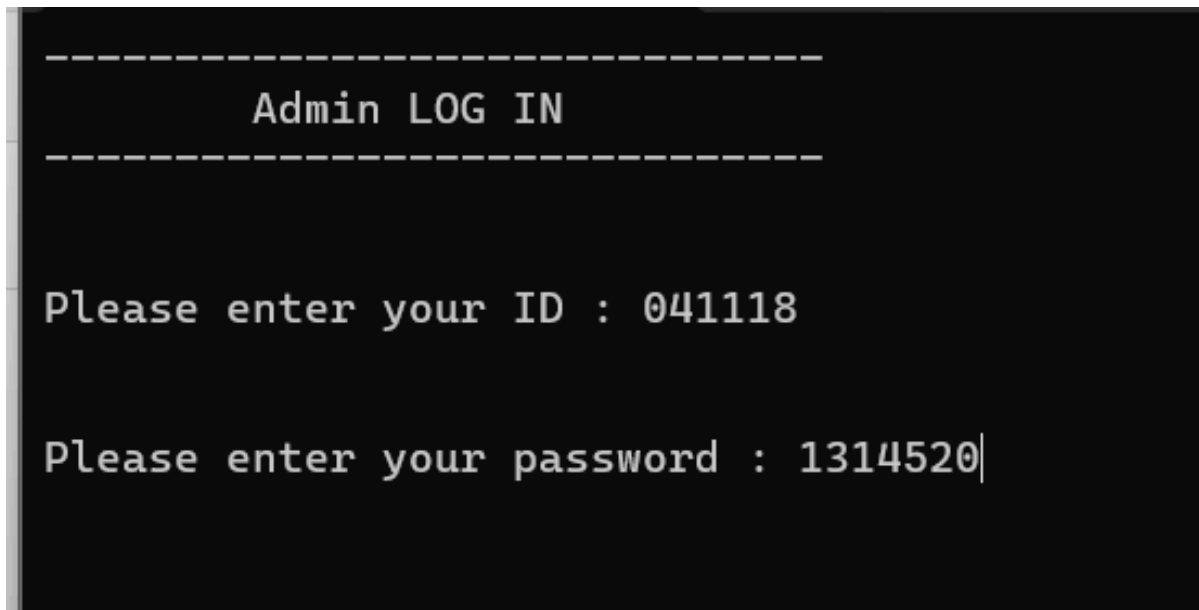


Screen 7: Customer Receipt

Screen 7: The receipt will be displayed with the discount given, amount the customer needed to pay before the discount given, the total payment the customer must pay. Other than that, the payment method which are three types, through bank account, TNG number or QR scanning will also be accepted.

ADMIN INTERFACE

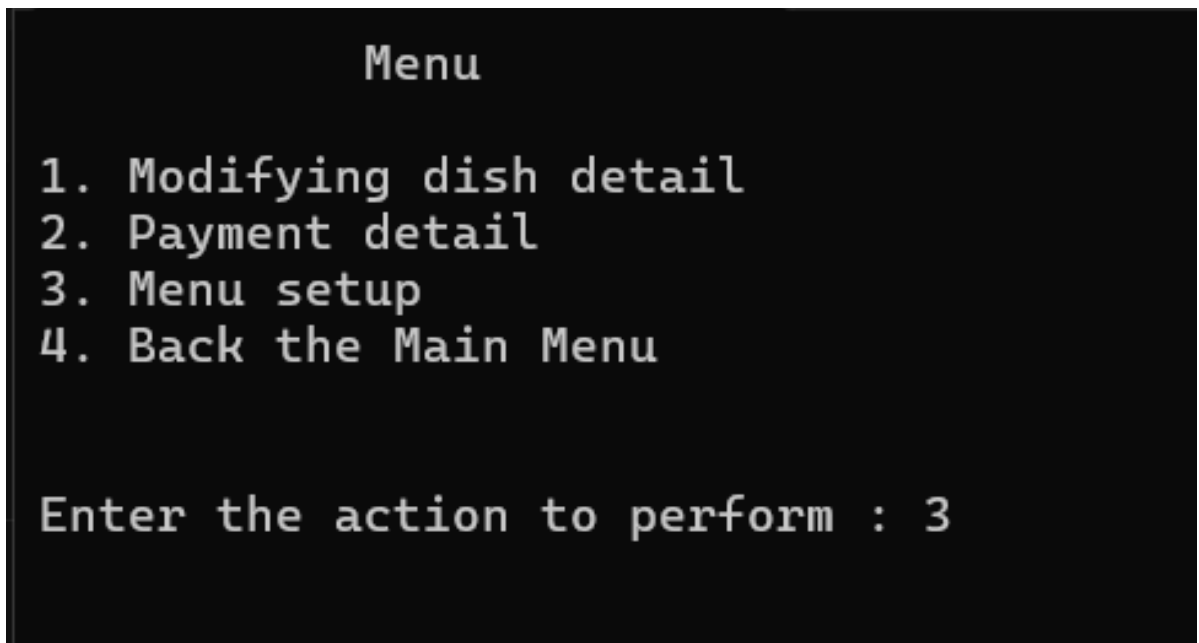
1.Admin Login Interface



Screen 1: Admin Login Interface

Screen 1: The admin is required to enter the correct password to login the merchant interface. After entering the correct password, the management interface will be shown. The admin ID and Password can be referred to the txt file adminList.txt and adminPass.txt .

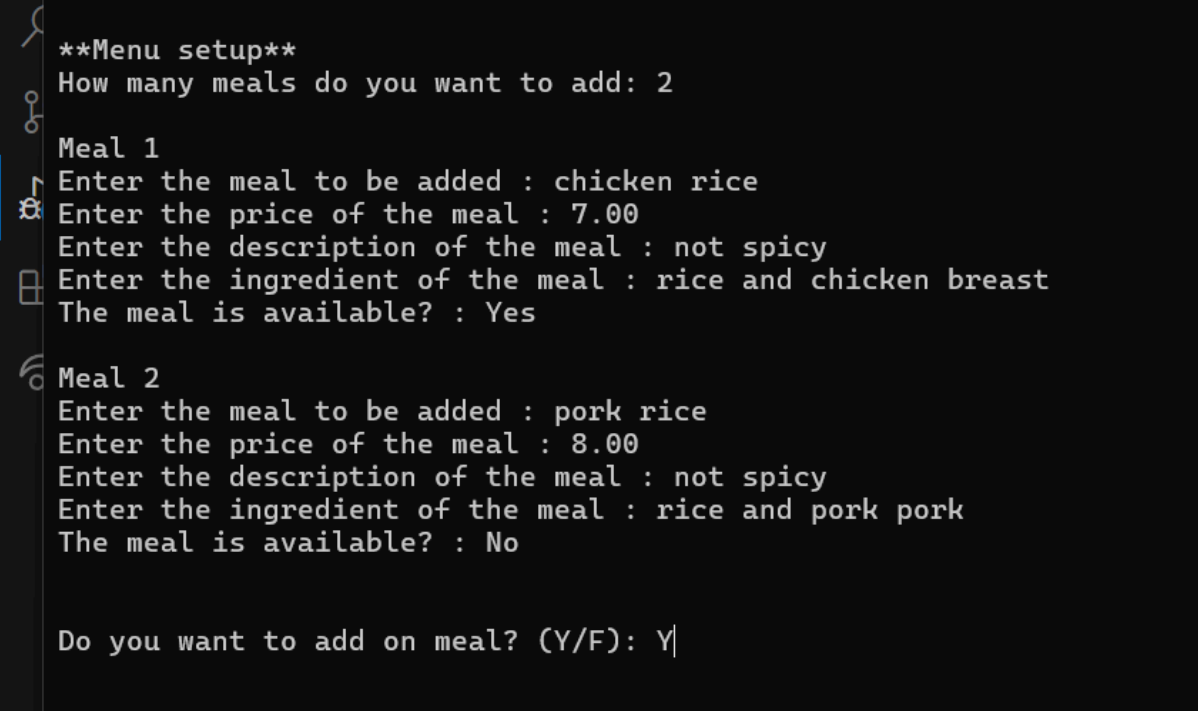
2. Management Menu



Screen 2: Management Menu

Screen 2: The screen 2 show that there are 4 types which are modifying the dish detail, modifying the payment detail and the menu setup and lastly go back to the main menu. Unless the correct input is given, then the system will pass to the next interface.

3. Menu Setup Interface

A screenshot of a terminal window with a dark background and light green text. On the left side, there is a vertical sidebar with several icons: a person, a magnifying glass, a list, a document, a folder, and a Wi-Fi symbol. The main area of the terminal displays the following text:

```
**Menu setup**
How many meals do you want to add: 2

Meal 1
Enter the meal to be added : chicken rice
Enter the price of the meal : 7.00
Enter the description of the meal : not spicy
Enter the ingredient of the meal : rice and chicken breast
The meal is available? : Yes

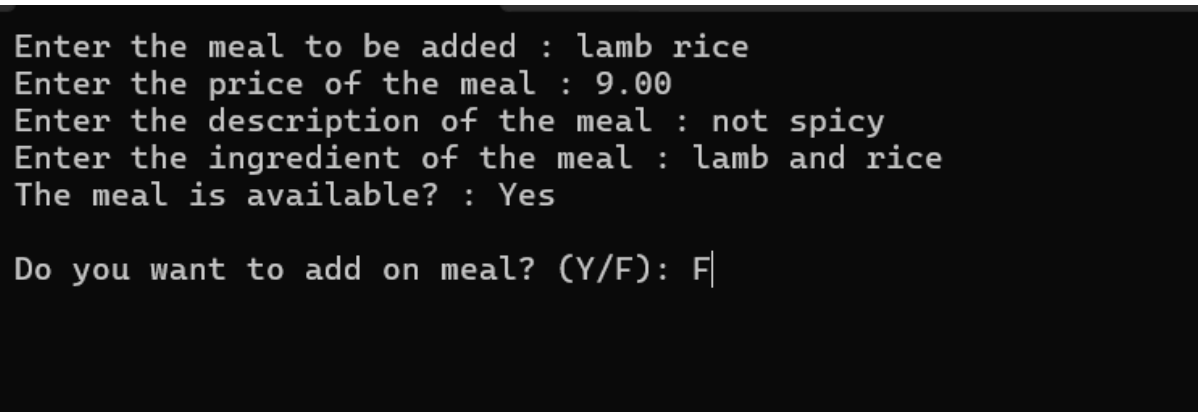
Meal 2
Enter the meal to be added : pork rice
Enter the price of the meal : 8.00
Enter the description of the meal : not spicy
Enter the ingredient of the meal : rice and pork pork
The meal is available? : No

Do you want to add on meal? (Y/F): Y|
```

Screen 3: Menu Setup Interface

Screen 3: When the admin entered 3, it will pass to the menu setup interface. The screen shows that the admin must set up the menu they have inside the system. The quantity, name, price, description, ingredient, and the availability of the dishes they need to update the latest in the system. After that, the admin can choose whether they want to add more meals or next to another interface to manage the admin panel.

4. Addition meal setup interface

A screenshot of a terminal window with a dark background and light green text. The main area of the terminal displays the following text:

```
Enter the meal to be added : lamb rice
Enter the price of the meal : 9.00
Enter the description of the meal : not spicy
Enter the ingredient of the meal : lamb and rice
The meal is available? : Yes

Do you want to add on meal? (Y/F): F|
```

Screen 4: Addition meal setup interface

Screen 4: The admin must enter the name, price, description, ingredient, and the availability of the dishes so that the menu will be updated. After that, the admin can choose whether they want to add more meals or next to another interface to manage the admin panel.

5. Updated Menu interface

```

**Updated Menu**
No.      Meal name      Price($)      Description      Ingredients      Availability
1        chicken rice    7             not spicy       rice and chicken breast    Yes
2        pork rice       8             not spicy       rice and pork pork        No
3        lamb rice       9             not spicy       lamb and rice             Yes

Enter (Y) to get back to main menu : |

```

Screen 5: Updated Menu interface

Screen 5: This interface will show the updated menu which is the latest one. Then, the admin must enter “Y” to get back to the main menu.

6. Modifying Meal Interface

```

*****
MODIFYING DISH DETAIL
*****
**Updated Menu**
No.      Meal name      Price($)      Description      Ingredients      Availability
1        chicken rice    7             not spicy       rice and chicken breast    Yes
2        pork rice       8             not spicy       rice and pork pork        No
3        lamb rice       9             not spicy       rice and lamb             Yes

Enter The Dish No. : 1

*****

Dish No. : 1
Dish Name : chicken rice
Price : 7
Available : Yes

*****
Which detail need to be edit?
Name(N)
Price(P)
Availability(A)

Enter your option -> N
Enter the meal to be added : CHICKEN RICE

Changed saved!
**Updated Menu**
No.      Meal name      Price($)      Description      Ingredients      Availability
1        CHICKEN RICE    7             not spicy       rice and chicken breast    Yes
2        pork rice       8             not spicy       rice and pork pork        No
3        lamb rice       9             not spicy       rice and lamb             Yes

Enter (Y) to get back to main menu :

```

Screen 6: Modifying Meal Interface

Screen 6 : When the admin entered 1, it will pass to the modifying meal interface. Before starting the modification of the meal , the system will show the latest menu. After that, the admin must enter the number system which is in the range of the meal. For example, we have only 10 types of meals, so the admin can only enter the number between 1 to 10. Besides, the admin must enter either N stand for name, P stand for price and A stand for availability to

modify the meal followed with the meal they have. After changing the menu, the updated menu will be shown again. Then, admin is required to enter Y to get back to the main menu.

7. Payment Interface

```
*****
      PAYMENT DETAIL
*****

Bank num : 24196210310
TNG num : 01116596391
QR CODE : https://nkf.org.my/wp-content/uploads/2020/10/tngo-ewallet.jpg

Which part need to edit?
Bank Num(B)
TNG Num(T)
QR Code(Q)

Enter your option ->B

Enter the new bank number : 8040808313

Enter (Y) to get back to main menu : |
```

Screen 7: Payment Interface (BANK NUMBER)

```

*****
PAYMENT DETAIL
*****

Bank num : 24196210310
TNG num : 01116596391
QR CODE : https://nkf.org.my/wp-content/uploads/2020/10/tngo-ewallet.jpg

Which part need to edit?
Bank Num(B)
TNG Num(T)
QR Code(Q)

Enter your option ->T

Enter the new TNG number : 016509369000

Enter (Y) to get back to main menu : Y

```

Screen 8: Payment Interface (TNG NUMBER)

```

*****
PAYMENT DETAIL
*****

Bank num : 24196210310
TNG num : 016509369000
QR CODE : https://nkf.org.my/wp-content/uploads/2020/10/tngo-ewallet.jpg

Which part need to edit?
Bank Num(B)
TNG Num(T)
QR Code(Q)

Enter your option ->Q

Enter the new path of QR code : https://nkf.org.my/wp-content/uploads/2020/10/tngo-ewallet.jpg

Enter (Y) to get back to main menu : Y|

```

Screen 9: Payment Interface(QR CODE)

Screen 7,8,9: When the admin entered 2, it will pass to the PAYMENT interface. The admin must enter whether B,T or Q. B stands for bank no, T stands for TNG no and Q stands for QR code. After entering their option, the new bank no, TNG no or QR code is required to enter to modify the payment interface.

PART 4: APPENDIX

Source of main function

```
1  #include <iostream>
2  #include <windows.h>
3  #include <iomanip>
4  #include <fstream>
5  #include "meal_detail.hpp"
6  #include "login_detail.hpp"
7  #include "payment_detail.hpp"
8  #include "CustomerType.hpp"
9  #define MAX_NUMBER 100
10 using namespace std;
11
12
13
14 int main(){
15     string descriptions,ingredients;
16     double prices;
17     int
18 num,loginAnswer,adminChoice,dishNo,customerChoice,numOrder=0,inde
19 x=0;
20
21     char ans = 'F';
22     string Menu;
```

```

23     Meal meals(num);
24     int order[MAX_NUMBER];
25     Payment pay;
26     Login user;
27     CustomerType type;
28     double sum=0,newSum;
29     char answerBack='P';
30
31
32
33 Login:
34 {
35     cout<<"-----"<<endl;
36     cout<<"  Restaurant ordering//payment system"<<endl;
37     cout<<"-----"<<endl;
38     cout<<"\n\n\t 1. Merchant"<<endl;
39     cout<<"\t 2. Customer"<<endl;
40     cout<<"\t 3. Exit"<<endl;
41     cout<<"\nEnter the action to perform : ";
42     cin>>loginAnswer;
43     cout<<endl;
44     system("cls");
45
46     switch (loginAnswer)
47     {
48     case 1:
49         goto adminLogin;
50         break;
51     case 2:
52         goto customerMenu;
53         break;
54     default:
55         goto end;
56     }
57
58     goto end;
59
60 //Log in as admin
61 adminLogin:
62 {
63     bool loginAnswer=false;
64     cout<<"-----"<<endl;
65
66
67
68
69
70
71

```

```

72         cout<<"          Admin LOG IN"<<endl;
73         cout<<"-----"<<endl;
74         user.userInput();
75         loginAnswer=user.adminValidityCheck();
76         if(!loginAnswer)
77         {
78             cout<<endl;
79             cout<<endl;
80             Sleep(2000);
81             system("cls");
82             goto adminLogin;
83         }
84         cout<<endl;
85     }
86     system("cls");
87     goto adminMenu;
88
89 adminMenu:
90 {
91     char answerBack='P';
92     cout<<right;
93     cout<<setw(15)<<"Menu"<<endl;
94     cout<<endl;
95     cout<<left;
96     cout<<"1. Modifying dish detail"<<endl;
97     cout<<"2. Payment detail"<<endl;
98     cout<<"3. Menu setup"<<endl;
99     cout<<"4. Back the Main Menu"<<endl;
100    cout<<"\n\nEnter the action to perform : ";
101    cin>>adminChoice;
102
103    switch (adminChoice)
104    {
105    case 1:
106        system("cls");
107        goto DishModify;
108    case 2:
109        system("cls");
110        goto PaymentDetail;
111    case 3:
112        system("cls");

```

```

121         goto MenuSetup;
122         break;
123     default:
124         system("cls");
125         break;
126     }
127 }
128 }
129
130
131 customerMenu:
132 {
133     char answerBack='P';
134     cout<<right;
135     cout<<setw(15)<<"Menu"<<endl;
136     cout<<endl;
137     cout<<left;
138     cout<<"1. Member Register"<<endl;
139     cout<<"2. Member login"<<endl;
140     cout<<"3. Making order"<<endl;
141     cout<<"4. Exit"<<endl;
142     cout<<"\n\nEnter the action to perform : ";
143     cin>>customerChoice;
144     system("cls");
145
146     switch (customerChoice)
147     {
148     case 1:
149         goto Register;
150         break;
151     case 2:
152         goto MemberLogin;
153         break;
154     case 3:
155         goto MakeOrder;
156         break;
157     default:
158         goto end;
159         break;
160     }
161 }
162
163 //Log in as member

```

```

170 MemberLogin:
171 {
172     bool loginAnswer=false;
173     cout<<"-----"<<endl;
174     cout<<"          Member LOG IN"<<endl;
175     cout<<"-----"<<endl;
176     user.userInput();
177     loginAnswer=user.customerValidityCheck();
178     if(!loginAnswer)
179     {
180         cout<<endl;
181         cout<<endl;
182         Sleep(2000);
183         system("cls");
184         goto MemberLogin;
185     }
186     Sleep(2000);
187     index=user.getIndex();
188     system("cls");
189 }
190 goto customerMenu;
191
192 Register:
193 {
194     cout<<"Register Section"<<endl;
195     cout<<"-----"<<endl;
196     cout<<endl;
197     user.userRegister();
198     system("cls");
199 }
200 goto customerMenu;
201
202 MakeOrder:
203 {
204     char orderChoice='Y';
205     int mealNo,mealNum;
206     for(int i=0;i<num;i++)
207     {
208         order[i]=0;
209     }
210     meals.displayMenu(num);
211     cout<<endl;
212

```

```

219     cout<<endl;
220     while (orderChoice=='Y')
221     {
222         numOrder++;
223         cout<<"\nEnter the meal you want to add : ";
224         cin>>mealNo;
225         if (meals.getAvailability (mealNo-1)=="No")
226         {
227             cout<<"This meal is currently not available."<<endl;
228             continue;
229         };
230         cout<<"How many do you want to order
231 " << meals.getMeal (mealNo-1) <<" : ";
232         cin>>mealNum;
233         order[mealNo-1]+=mealNum;
234         cout<<"Do you want to add on something (Y)/(N) : ";
235         cin>>orderChoice;
236     }
237     system("cls");
238 }
239
240 goto MealChecking;
241
242
243
244
245
246
247 MealChecking:
248 {
249     int checkingOption;
250     string memberType;
251     sum=0;
252     cout<<"*****"<<endl;
253     cout<<"          Overview Order"<<endl;
254     cout<<"*****"<<endl;
255
256     cout<<"\n\n"<<setw(5)<<"No."<<setw(20)<<"Meal"<<setw(20)<<"Quantity"<<endl;
257     memberType=type.getType (index) ;
258     for (int i=0;i<=num;i++)
259     {
260         if (order[i] !=0)
261         {
262             cout<<setw(5)<<i+1<<setw(20)<<meals.getMeal (i) <<setw(20)<<order[i

```

```

268 ]<<endl;
269         sum+=order[i]*meals.getPrice(i);
270     }
271 }
272
273 double discount=0;
274 cout<<"\n\nEnter 1 : Proceed to Payment "<<endl;
275 cout<<"Enter 2 : Exit"<<endl;
276 cout<<"\n\nEnter the action to perform : ";
277 cin>>checkingOption;
278 system("cls");
279 if(checkingOption==1)
280 {
281     cout<<"\n\n\n*****"<<endl;
282     cout<<"          Reciept"<<endl;
283     cout<<"*****"<<endl;
284     cout<<endl;
285     cout<<endl;
286     cout<<"Discount =
287 "<<((1-type.getDiscount())*100)<<"%"<<"\nAmount before Discount =
288 "<<sum<<endl;
289     newSum=sum*type.getDiscount();
290     cout<<"Total payment : RM "<<newSum<<endl;
291     cout<<"\nYou are log in as "<<memberType<<"
292 customer"<<endl;
293     cout<<"\n\n\n\n"<<endl;
294     pay.displayDetail();
295     cout<<endl;
296 }else{
297     system("cls");
298     goto customerMenu;
299 }
300
301 }
302
303 }
304
305
306 goto end;
307
308
309
310 MenuSetup:
311 {
312     cout << "\n\n**Menu setup**" << endl;
313     cout << "How many meals do you want to add: ";
314     cin >> num;
315     cin.ignore();
316

```

```

317     cout << endl;
318     for (int i = 0; i < num; i++) {
319         cout << "Meal " << i + 1 << endl;
320         meals.setMeal(i);
321         meals.setPrice(i);
322         meals.setDescription(i);
323         meals.setIngredient(i);
324         meals.setAvailability(i);
325         cout << endl;
326     }
327     cout << endl;
328     cout << "Do you want to add on meal? (Y/F): ";
329     cin >> ans;
330     cin.ignore();
331     system("cls");
332     while (ans == 'Y') {
333         meals.setMeal(num);
334         meals.setPrice(num);
335         meals.setDescription(num);
336         meals.setIngredient(num);
337         meals.setAvailability(num);
338         num++;
339         cout<<endl;
340         cout << "Do you want to add on meal? (Y/F): ";
341         cin >> ans;
342         cin.ignore();
343         system("cls");
344     }
345     cout << endl;
346     meals.displayMenu(num);
347     cout<<endl;
348     cout<<endl;
349     cout<<"Enter (Y) to get back to main menu : ";
350     cin>>answerBack;
351     if(answerBack=='Y')
352     {
353         system("cls");
354         goto adminMenu;
355     }else{
356         system("cls");
357         goto end;
358     }
359

```



```

366     }
367
368
369     goto adminMenu;
370
371 DishModify:
372     {
373         char option;
374         cout<<"*****"<<endl;
375         cout<<"    MODIFYING DISH DETAIL"<<endl;
376         cout<<"*****"<<endl;
377         meals.displayMenu(num);
378         cout<<"\n\nEnter The Dish No. : ";
379         cin>>dishNo;
380         cout<<"\n\n*****"<<endl;
381         cout<<"\n\nDish No. : "<<dishNo<<endl;
382         cout<<"Dish Name : "<<meals.getMeal(dishNo-1)<<endl;
383         cout<<"Price : "<<meals.getPrice(dishNo-1)<<endl;
384         cout<<"Available : "<<
385         meals.getAvailability(dishNo-1)<<endl;
386         cout<<"\n\n*****"<<endl;
387         cout<<"\tWhich detail need to be edit?"<<endl;
388         cout<<"\t\tName(N) "<<endl;
389         cout<<"\t\tPrice(P) "<<endl;
390         cout<<"\t\tAvailability(A) "<<endl;
391         cout<<"\n\nEnter your option -> ";
392         cin>>option;
393         cin.ignore();
394
395         switch (option)
396         {
397             case 'N':
398                 meals.setMeal(dishNo-1);
399                 break;
400             case 'P':
401                 meals.setPrice(dishNo-1);
402                 break;
403             case 'A':
404                 meals.setAvailability(dishNo-1);
405                 break;
406             default:
407                 break;
408         }
409     }
410
411
412
413
414

```

```

415     }
416     cout<<endl;
417     cout<<"Changed saved!"<<endl;
418     meals.displayMenu(num);
419     cout<<endl;
420     cout<<endl;
421     cout<<"Enter (Y) to get back to main menu : ";
422     cin>>answerBack;
423     if(answerBack=='Y')
424     {
425         goto adminMenu;
426     }else{
427         goto end;
428     }
429     }
430     }
431     }
432     goto adminMenu;
433
434 PaymentDetail:
435 {
436     char paymentOption;
437     pay.displayDetail();
438     cout<<"\n\n\nWhich part need to edit?"<<endl;
439     cout<<"Bank Num(B) "<<endl;
440     cout<<"TNG Num(T) "<<endl;
441     cout<<"QR Code(Q) "<<endl;
442     cout<<"\n\nEnter your option ->";
443     cin>>paymentOption;
444     switch (paymentOption)
445     {
446     case 'B':
447         pay.setBank();
448         break;
449     case 'T':
450         pay.setTNG();
451         break;
452     case 'Q':
453         pay.setQR();
454         break;
455     default:
456         break;
457     }
458
459
460
461
462
463

```

```

464         cout<<endl;
465         cout<<endl;
466         cout<<"Enter (Y) to get back to main menu : ";
467         cin>>answerBack;
468         system("cls");
469         if(answerBack=='Y')
470         {
471             goto adminMenu;
472         }else{
473             goto end;
474         }
475     }
476 }
477 goto adminMenu;
478 end:
479 {
480     system("pause");
481     return 0;
482 }
483 }
484 }
485 }
486

```

Source of user cpp

```

1  #include <iostream>
2  #include "user.hpp"
3  using namespace std;
4
5
6
7  User::User(int user, int pass): username(user), password(pass)
8  {}
9  bool User::validate(){
10     return false;
11 }
12
13
14 int User::getIndex(){
15     return 0;
16 }

```

Source of user.hpp

```
1  #ifndef USER_H
2  #define USER_H
3
4  #include <string>
5  using namespace std;
6
7
8  class User {
9  protected:
10     int username;
11     int password;
12
13
14 public:
15     User(int user, int pass);
16     virtual bool validate();
17     virtual int getIndex();
18     virtual ~User() {}
19 };
20
21
22 #endif
23
24
```

Source of admin.hpp

```
1  #ifndef ADMIN_H
2  #define ADMIN_H
3
4  #include "user.hpp"
5  #include <fstream>
6
7
8  class Admin : public User {
9  public:
10     Admin(int user, int pass):User(user,pass){}
11     bool validate() override{
12         bool validityID=false,validityPass=false;
13         fstream adminID("adminList.txt",ios::in);
14         fstream adminPass("adminPass.txt",ios::in);
15         int tempID,tempPass,num=0;
16         while (!adminID.eof())
17         {
18
19
```

```

20         adminID>>tempID;
21         if (tempID==this->username)
22         {
23             validityID=true;
24             break;
25         }
26         num++;
27     }
28
29
30     if(validityID)
31     {
32         for(int i=0;i<=num;i++)
33         {
34             adminPass>>tempPass;
35         }
36         if (tempPass==this->password)
37         {
38             validityPass=true;
39         }else{
40             cout<<"Wrong userID/Password!"<<endl;
41             return false;
42         }
43     }else{
44         cout<<"Wrong userID/Password!"<<endl;
45         return false;
46     }
47
48     if(validityPass==true)
49     {
50         cout<<"Success Login"<<endl;
51         return true;
52     }
53
54     return false;
55
56 };
57
58 #endif
59
60
61
62
63
64
65
66
67
68

```

Source of customer.cpp

```
1  #include <iostream>
2  #include <fstream>
3  #include "customer.hpp"
4  #include "user.hpp"
5  using namespace std;
6
7
8  void Customer::setIndex(int num)
9  {
10     this->userIndex=num;
11 }
12
```

Source of customer.hpp

```
1  #ifndef CUSTOMER_H
2  #define CUSTOMER_H
3  #include <fstream>
4  #include "user.hpp"
5
6
7  class Customer : public User {
8  private:
9      int userIndex;
10 public:
11     void setIndex(int) ;
12     int getIndex() override{
13         return userIndex;
14     }
15
16     Customer(int user, int pass) : User(user,pass){}
17     bool validate() override{
18         bool validityID=false,validityPass=false;
19         fstream customerID("customerList.txt",ios::in) ;
20         fstream customerPass("customerPass.txt",ios::in) ;
21         int tempID,tempPass,num=0;
22
23
```

```

24     while (!customerID.eof())
25     {
26         customerID>>tempID;
27         if (tempID==this->username)
28         {
29             validityID=true;
30             break;
31         }
32         num++;
33     }
34
35
36     if(validityID)
37     {
38         for(int i=0;i<=num;i++)
39         {
40             customerPass>>tempPass;
41         }
42         if (tempPass==this->password)
43         {
44             validityPass=true;
45         }else{
46             cout<<"Wrong userID/Password!"<<endl;
47             return false;
48         }
49     }else{
50         cout<<"Wrong userID/Password!"<<endl;
51         return false;
52     }
53
54     if(validityPass==true)
55     {
56         cout<<"Success Login"<<endl;
57         setIndex(num) ;
58         return true;
59     }
60     return false;
61 }
62 };
63
64 #endif
65
66
67
68
69
70

```

Source of customerType.cpp

```
1  #include <iostream>
2  #include <fstream>
3  #include "CustomerType.hpp"
4  using namespace std;
5
6
7  double checkDiscount(string type)
8  {
9      double discounts;
10     if (type == "Regular")
11         discounts= 1;
12     else if (type == "Premium")
13         discounts= 0.9;
14     else if (type == "VIP")
15         discounts= 0.85;
16
17     return discounts;
18 }
19
20
21
22 CustomerType::CustomerType() : type("Regular") {}
23
24 string CustomerType::getType(int index) {
25     if(index==0)
26     {
27
28         cout<<"You login as a guest"<<endl;
29         return "Guest";
30     }
31     ifstream typeCustomer("customerType.txt", ios::in);
32     if (!typeCustomer) {
33         cerr << "Error opening file." << endl;
34         return "";
35     }
36
37
38     string tempType;
39     for (int i = 0; i <=index; i++) {
40         getline(typeCustomer,tempType);
41     }
42     this->type = tempType;
43     return this->type;
44 }
45
46
47
```



```

48 double CustomerType::getDiscount()
49 {
50     discount=checkDiscount(this->type);
51     return discount;
52 }
53

```

Source of customerType.hpp

```

1  #ifndef CUSTOMERTYPE_H
2  #define CUSTOMERTYPE_H
3
4  #include <string>
5  #include <fstream>
6  using namespace std;
7  class CustomerType {
8  private:
9
10     string type;
11     double discount; // Discount percentage
12
13 public:
14     CustomerType();
15
16     string getType(int);
17     double getDiscount();
18 };
19
20
21
22 #endif

```

Source of login_detail.cpp

```

1  #include <iostream>
2  #include <fstream>
3  #include "login_detail.hpp"
4  using namespace std;
5
6  Login::Login()
7  {
8      userID=0;
9      userPassword=0;
10 }
11
12 void Login::userInput()
13 {
14     int id,pass;
15     cout<<"\n\nPlease enter your ID : ";
16     cin>>id;
17     cin.ignore();
18     userID=id;
19     cout<<"\n\nPlease enter your password : ";
20     cin>>pass;
21     cin.ignore();
22     cout<<endl;
23     cout<<endl;
24     userPassword=pass;
25 }
26
27 bool Login::adminValidityCheck() const
28 {
29     bool validityID=false,validityPass=false;
30     fstream adminID("adminList.txt",ios::in);
31     fstream adminPass("adminPass.txt",ios::in);
32     int tempID,tempPass,num=0;
33     while (!adminID.eof())
34     {
35         adminID>>tempID;
36         if(tempID==userID)
37         {
38             validityID=true;
39             break;
40         }
41         num++;
42     }
43 }

```

```

49     }
50
51     if(ValidityID)
52     {
53         for(int i=0;i<=num;i++)
54         {
55             adminPass>>tempPass;
56         }
57         if(tempPass==userPassword)
58         {
59             ValidityPass=true;
60         }else{
61             cout<<"Wrong userID/Password!"<<endl;
62             return false;
63         }
64     }else{
65         cout<<"Wrong userID/Password!"<<endl;
66         return false;
67     }
68
69     if(ValidityPass==true)
70     {
71         cout<<"Success Login"<<endl;
72         return true;
73     }
74     return false;
75 }
76
77 bool Login::customerValidityCheck()
78 {
79     bool ValidityID=false,ValidityPass=false;
80     fstream customerID("customerList.txt",ios::in);
81     fstream customerPass("customerPass.txt",ios::in);
82     int tempID,tempPass,num=0;
83     while (!customerID.eof())
84     {
85         customerID>>tempID;
86         if(tempID==userID)
87         {
88             ValidityID=true;
89             break;
90         }
91     }
92 }

```

```

98         num++;
99     }
100
101     if(ValidityID)
102     {
103         for(int i=0;i<=num;i++)
104         {
105             customerPass>>tempPass;
106         }
107         if(tempPass==userPassword)
108         {
109             ValidityPass=true;
110         }else{
111             cout<<"Wrong userID/Password!"<<endl;
112             return false;
113         }
114     }else{
115         cout<<"Wrong userID/Password!"<<endl;
116         return false;
117     }
118
119     if(ValidityPass==true)
120     {
121         cout<<"Success Login"<<endl;
122         setIndex(num);
123         return true;
124     }
125     return false;
126 }
127
128 void Login::userRegister()
129 {
130     int phone, password;
131     string memberTypes="Premium";
132     ofstream userID("customerList.txt", ios::app);
133     ofstream userPass("customerPass.txt", ios::app);
134     ofstream memberType("CustomerType.txt", ios::app);
135     cout << "Please enter your phone number : ";
136     cin >> phone;
137     userID << phone << endl;
138     cout << "\nPlease enter your password : ";
139     cin >> password;

```

```

147     userPass << password << endl;
148     cout<<endl;
149     memberType << memberTypes << endl;
150     cout << "Successfully registered! :)" << endl;
151 }
152
153
154 void Login::setIndex(int num)
155 {
156     userIndex=num;
157 }
158
159
160 int Login::getIndex() const
161 {
162     return userIndex;
163 }
164

```

Source of login_detail.hpp

```

1  #ifndef LOGIN_DETAIL_H
2  #define LOGIN_DETAIL_H
3  using namespace std;
4
5  class Login{
6      private:
7          int userID;
8          int userPassword;
9          int userIndex;
10     public:
11         Login() ;
12
13         //mutator method
14         void userInput() ;
15         bool adminValidityCheck() const;
16         bool customerValidityCheck() ;
17         void userRegister() ;
18         void setIndex(int) ;
19         int getIndex() const;
20 };
21
22
23
24
25

```

Source of meal_detail.cpp

```
1  #include <iostream>
2  #include <string>
3  #include <iomanip>
4  #include "meal_detail.hpp"
5  using namespace std;
6
7
8  Meal::Meal(int num):number(num){}
9  void Meal::setMeal(int num)
10 {
11     string meals;
12     cout<<"Enter the meal to be added : ";
13     getline(cin,meals);
14     Menu[num]=meals;
15 }
16
17
18 void Meal::setPrice(int num)
19 {
20     double prices;
21     cout<<"Enter the price of the meal : ";
22     cin>>prices;
23     price[num]=prices;
24     cin.ignore();
25 }
26
27
28 void Meal::setDescription(int num)
29 {
30     string descriptions;
31     cout<<"Enter the description of the meal : ";
32     getline(cin,descriptions);
33     description[num]=descriptions;
34 }
35
36
37
38 void Meal::setIngredient(int num)
39 {
40     string ingredients;
41     cout<<"Enter the ingredient of the meal : ";
42     getline(cin,ingredients);
43 }
44
```

```

45     ingredient[num]=ingredients;
46 }
47
48 void Meal::setAvailability(int num)
49 {
50     string available;
51     cout<<"The meal is available? : ";
52     getline(cin,available);
53     availability[num]=available;
54 }
55
56
57
58
59 string Meal::getMeal(int num) const
60 {
61     return Menu[num];
62 }
63
64
65 string Meal::getDescription(int num) const
66 {
67     return description[num];
68 }
69
70
71 string Meal::getIngredient(int num) const
72 {
73     return ingredient[num];
74 }
75
76
77 double Meal::getPrice(int num) const
78 {
79     return price[num];
80 }
81
82
83 string Meal::getAvailability(int num) const
84 {
85     return availability[num];
86 }
87
88 void Meal::displayMenu(int num) const
89 {
90     string menu,descriptions,ingredients,availabilities;
91     double prices;
92     cout << "***Updated Menu**" << endl;
93

```

```

94     cout << left;
95     cout << setw(10) << "No." << setw(20) << "Meal name" <<
96     setw(15) << "Price($)"
97     << setw(30) << "Description" << setw(30) << "Ingredients" << setw(20) << "A
98     vailability" << endl;
99
100    for (int i = 0; i < num; i++) {
101        menu=Menu[i];
102        prices = price[i];
103        descriptions = description[i];
104        ingredients = ingredient[i];
105        availabilities = availability[i];
106        cout << left;
107        cout << setw(10) << i + 1 << setw(20) << menu <<
108        setw(15) << prices
109        << setw(30) << descriptions << setw(30) << ingredients << setw(20) << avail
110        abilities << endl;
111    }
112
113 }
114

```

Source of meal_detail.hpp

```

1  #ifndef MEAL_DETAIL_H
2  #define MEAL_DETAIL_H
3
4  #define MAX_NUMBER 100
5  #include <string>
6  using namespace std;
7
8  class Meal
9  {
10
11     private:
12         string Menu[MAX_NUMBER];
13         double price[MAX_NUMBER];
14         string description[MAX_NUMBER];
15         string ingredient[MAX_NUMBER];
16         string availability[MAX_NUMBER];
17         int number;
18     public:
19         Meal(int num);
20
21
22         //mutator method
23         void setMeal(int num);
24

```



```

25     void setPrice(int num) ;
26     void setDescription(int num) ;
27     void setIngredient(int num) ;
28     void setAvailability(int num) ;
29
30     //accessor method
31     string getMeal(int num) const;
32     double getPrice(int num) const;
33     string getDescription(int num) const;
34     string getIngredient(int num) const;
35     string getAvailability(int num) const;
36     void displayMenu(int num) const;
37 };
38
39 #endif
40
41

```

Source of payment_detail.cpp

```

1  #include <iostream>
2  #include "payment_detail.hpp"
3  using namespace std;
4
5  Payment::Payment()
6  {
7      bankNum="24196210310";
8      tngNum="01116596391";
9
10     qrPath="https://nkf.org.my/wp-content/uploads/2020/10/tngo-ewallet.jpg";
11
12 }
13
14
15 void Payment::setBank()
16 {
17     cout<<"\nEnter the new bank number : ";
18     cin>>bankNum;
19     cout<<endl;
20
21
22
23

```

```

24     cin.clear();
25 }
26
27 void Payment::setTNG()
28 {
29     cout<<"\nEnter the new TNG number : ";
30     cin>>tngNum;
31     cout<<endl;
32     cin.clear();
33 }
34
35
36 void Payment::setQR()
37 {
38     cout<<"\nEnter the new path of QR code : ";
39     cin>>qrPath;
40     cout<<endl;
41     cin.clear();
42 }
43
44
45
46 string Payment::getBank() const
47 {
48     return bankNum;
49 }
50
51
52 string Payment::getTNG() const
53 {
54     return tngNum;
55 }
56
57
58 string Payment::getQR() const
59 {
60     return qrPath;
61 }
62
63
64 void Payment::displayDetail() const
65 {
66     string bank,tng,qr;
67     bank=getBank();
68     tng=getTNG();
69     qr=getQR();
70     cout<<endl;
71     cout<<"*****"<<endl;
72

```

73	<code>cout<<" PAYMENT DETAIL"<<endl;</code>
74	<code>cout<<"*****"<<endl;</code>
75	<code>cout<<"\n\nBank num : "<<bank<<endl;</code>
76	<code>cout<<"TNG num : "<<tng<<endl;</code>
77	<code>cout<<"QR CODE : "<<qrPath<<endl;</code>
78	<code>system("start</code>
79	<code>https://nkf.org.my/wp-content/uploads/2020/10/tngo-ewallet.jpg"</code>
80	<code>);</code>
81	<code>}</code>
82	

Source of payment_detail.hpp

1	<code>#ifndef PAYMENT_DETAIL_H</code>
2	<code>#define PAYMENT_DETAIL_H</code>
3	<code>#include <string></code>
4	<code>using namespace std;</code>
5	
6	
7	<code>class Payment</code>
8	<code>{</code>
9	<code>private:</code>
10	<code>string bankNum;</code>
11	<code>string tngNum;</code>
12	<code>string qrPath;</code>
13	<code>public:</code>
14	<code>Payment();</code>
15	<code>void setBank();</code>
16	<code>void setTNG();</code>
17	<code>void setQR();</code>
18	<code>string getBank() const;</code>
19	<code>string getTNG() const;</code>
20	<code>string getQR() const;</code>
21	<code>void displayDetail() const;</code>
22	<code>};</code>
23	
24	<code>#endif</code>
25	
26	
27	

Source of adminList.txt

1	041118
2	011233
3	040131

Source of adminPass.txt

1	1314520
2	5201314
3	085757

Source of customerList.txt

1	041118
2	011233
3	073061
4	1234
5	9876
6	4567
7	3456
8	897897
9	6391
10	5059
11	8090

Source of customerPass.txt

1	520
2	1314
3	7181
4	4321
5	1234
6	7654
7	6789
8	1234
9	1234
10	6161
11	1234

Source of customerType.txt

1	Regular
2	Regular
3	Premium
4	Premium
5	Premium
6	Premium
7	Premium

8	Premium
9	VIP
10	Premium
11	Premium